

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Тульский государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №8
по дисциплине
Методы и средства защиты информации
по теме
«Алгоритм шифрования RSA»

Выполнил: ст. гр. 220032-11

Лаукерт К. М.

Принял: к.т.н., доцент ИПМКН

Кулешова Н. В.

Тула 2026

Лабораторная работа №8

по теме «Алгоритм шифрования RSA»

Цель работы

Получить навыки программной реализации модифицированного алгоритма RSA: сформировать пару ключей, зашифровать и расшифровать текст, а также создать и проверить электронную цифровую подпись.

Постановка задачи

Разработать визуальное приложение, которое проверяет простоту введённых p и q , формирует открытый и закрытый ключи RSA, выполняет шифрование текста по модифицированному алгоритму из методички, расшифровывает результат, формирует ЭЦП по ФИО и подписывает шифрованное сообщение.

Краткие теоретические сведения

RSA — асимметричный криптографический алгоритм. Для двух различных простых чисел p и q вычисляются $n = p \cdot q$ и $\phi(n) = (p-1)(q-1)$. Открытая экспонента e выбирается взаимно простой с $\phi(n)$, а закрытая экспонента d удовлетворяет условию $d \cdot e \equiv 1 \pmod{\phi(n)}$. Открытый ключ имеет вид (e, n) , закрытый — (d, n) . Шифрование блока m выполняется по формуле $c = m^e \bmod n$, расшифрование — $m = c^d \bmod n$.

В модифицированном варианте перед возведением блоков в степень используется цепочка: в начало добавляется случайное число, а очередной блок равен сумме исходного байта и предыдущего блока по модулю n . Поэтому одинаковые символы не имеют постоянного шифроблока и одинаковый текст при повторном запуске даёт иной шифртекст. При расшифровании цепочка восстанавливается вычитанием предыдущего блока.

Для ЭЦП сначала вычисляется SHA-256 от подписываемых данных. Представитель хеша modulo n возводится в степень d : $s = h^d \bmod n$. Подлинность проверяется сравнением $h \bmod n$ и $s^e \bmod n$. SHA-256 используется как стандартная односторонняя хеш-функция фиксированной длины; малое изменение исходных данных приводит к иному хешу.

Реализация

Приложение выполнено на Python с использованием стандартной библиотеки Tkinter. Сторонние криптографические библиотеки не применялись. Реализованы: проверка простоты делением до квадратного корня, расширенный алгоритм Евклида для нахождения обратного элемента, модифицированное RSA-шифрование UTF-8, обратное расшифрование и RSA-подпись SHA-256.

Параметр	Значение в демонстрации
р, q	10007 и 10009 — различные простые числа
Модуль n	100160063
Открытый ключ	(e = 65537, n = 100160063)
Закрытый ключ	(d = 35910881, n = 100160063)
Подписываемые данные	ФИО «Лаукерт Кирилл Максимович» и сериализованный шифртекст
Хеш-функция	SHA-256, представитель хеша берётся по модулю n

Выбранные в демонстрации небольшие числа позволяют наглядно проследить работу алгоритма. Они не предназначены для защиты реальных данных: для практического RSA применяют существенно большие криптографически стойкие простые числа и безопасные схемы дополнения.

Текст программы (rsa_lab.py):

```
"""Лабораторная работа №8: модифицированный RSA и ЭЦП.
```

```
Учебная реализация без сторонних криптографических библиотек.
Малые ключи
```

```
предназначены только для демонстрации алгоритмов и не подходят
для защиты
```

```
реальных данных.
```

```
"""
```

```
from __future__ import annotations
```

```
import hashlib
```

```

import json
import math
import secrets
import sys
import tkinter as tk
from dataclasses import dataclass
from tkinter import messagebox, ttk

@dataclass(frozen=True)
class KeyPair:
    """Пара открытого и закрытого ключей RSA."""

    n: int
    e: int
    d: int

def is_prime(number: int) -> bool:
    """Проверка простоты делением на нечётные делители до
    sqrt(number)."""
    if number < 2:
        return False
    if number in (2, 3):
        return True
    if number % 2 == 0:
        return False

    divisor = 3
    while divisor * divisor <= number:
        if number % divisor == 0:
            return False
        divisor += 2
    return True

```

```

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    gcd, x1, y1 = extended_gcd(b, a % b)
    return gcd, y1, x1 - (a // b) * y1

def mod_inverse(value: int, modulus: int) -> int:
    gcd, coefficient, _ = extended_gcd(value, modulus)
    if gcd != 1:
        raise ValueError("Не удалось найти обратный элемент.")
    return coefficient % modulus

def choose_public_exponent(phi: int) -> int:
    """Выбирает e, взаимно простое с  $\phi(n)$ ."""
    if 1 < 65537 < phi and math.gcd(65537, phi) == 1:
        return 65537
    for exponent in range(3, phi, 2):
        if math.gcd(exponent, phi) == 1:
            return exponent
    raise ValueError("Не удалось подобрать открытую экспоненту.")

def build_keys(p: int, q: int) -> KeyPair:
    if not is_prime(p):
        raise ValueError("Число p не является простым.")
    if not is_prime(q):
        raise ValueError("Число q не является простым.")
    if p == q:
        raise ValueError("Числа p и q должны различаться.")

    n = p * q
    if n <= 511:

```

```

        raise ValueError("Произведение  $p \cdot q$  должно быть больше
511 для кодирования UTF-8.")
    phi = (p - 1) * (q - 1)
    e = choose_public_exponent(phi)
    return KeyPair(n=n, e=e, d=mod_inverse(e, phi))

```

```

def encrypt_modified_rsa(text: str, public_key: KeyPair) ->
list[int]:

```

```

    """Шифрует UTF-8 по модифицированному алгоритму из
методички.

```

```

    Перед каждым байтом добавляется предыдущее число по модулю
n. В начале

```

```

    цепочки располагается случайный элемент, поэтому одинаковые
сообщения

```

```

    каждый раз дают разный набор шифроблоков.

```

```

    """

```

```

    source = text.encode("utf-8")

```

```

    if not source:

```

```

        raise ValueError("Введите текст для шифрования.")

```

```

    seed = secrets.randbelow(public_key.n - 256) + 256

```

```

    chained = [seed]

```

```

    previous = seed

```

```

    for byte in source:

```

```

        current = (previous + byte) % public_key.n

```

```

        chained.append(current)

```

```

        previous = current

```

```

    return [pow(block, public_key.e, public_key.n) for block in
chained]

```

```

def decrypt_modified_rsa(ciphertext: list[int], private_key:
KeyPair) -> str:

```

```

    """Расшифровывает и распутывает цепочку модифицированного
RSA."""

```

```

    if len(ciphertext) < 2:
        raise ValueError("Шифртекст не содержит данных.")

    chained = [pow(block, private_key.d, private_key.n) for
block in ciphertext]
    previous = chained[0]
    decoded = bytearray()
    for current in chained[1:]:
        byte = (current - previous) % private_key.n
        if byte > 255:
            raise ValueError("Шифртекст повреждён или выбран
неверный закрытый ключ.")
        decoded.append(byte)
        previous = current

    try:
        return decoded.decode("utf-8")
    except UnicodeDecodeError as error:
        raise ValueError("Шифртекст не соответствует UTF-8
сообщению.") from error

def message_hash(data: bytes, modulus: int) -> tuple[str, int]:
    digest = hashlib.sha256(data).hexdigest()
    return digest, int(digest, 16) % modulus

def sign(data: bytes, private_key: KeyPair) -> tuple[str, int,
int]:
    """Возвращает SHA-256, его представителя modulo n и RSA-
подпись."""
    digest, representative = message_hash(data, private_key.n)
    return digest, representative, pow(representative,
private_key.d, private_key.n)

def verify(data: bytes, signature: int, public_key: KeyPair) ->
bool:

```

```

_, representative = message_hash(data, public_key.n)
return pow(signature, public_key.e, public_key.n) ==
representative

```

```

class RsaLabApp(tk.Tk):
    def __init__(self) -> None:
        super().__init__()
        self.title("ЛР №8 – RSA: шифрование и ЭЦП")
        self.minsize(940, 700)
        self.key_pair: KeyPair | None = None

        self.p_var = tk.StringVar(value="10007")
        self.q_var = tk.StringVar(value="10009")
        self.fio_var = tk.StringVar(value="Лаукерт Кирилл  
Максимович")
        self.decrypted_var = tk.StringVar()

        self._build_interface()

    def _build_interface(self) -> None:
        root = ttk.Frame(self, padding=12)
        root.pack(fill=tk.BOTH, expand=True)
        root.columnconfigure(0, weight=1)
        root.rowconfigure(3, weight=1)

        ttk.Label(
            root,
            text="Модифицированный алгоритм RSA и электронная  
цифровая подпись",
            font=("Segoe UI", 14, "bold"),
        ).grid(row=0, column=0, sticky="w", pady=(0, 10))

        key_frame = ttk.LabelFrame(root, text="1. Параметры и  
ключи RSA", padding=10)
        key_frame.grid(row=1, column=0, sticky="ew", pady=(0,
10))

```

```

        for column in (1, 3, 5):
            key_frame.columnconfigure(column, weight=1)

            ttk.Label(key_frame, text="Простое p:").grid(row=0,
column=0, sticky="w")
            ttk.Entry(key_frame, textvariable=self.p_var,
width=16).grid(row=0, column=1, sticky="ew", padx=(6, 16))
            ttk.Label(key_frame, text="Простое q:").grid(row=0,
column=2, sticky="w")
            ttk.Entry(key_frame, textvariable=self.q_var,
width=16).grid(row=0, column=3, sticky="ew", padx=(6, 16))
            ttk.Button(key_frame, text="Сформировать ключи",
command=self.generate_keys).grid(row=0, column=4, columnspan=2,
sticky="e")

            self.keys_label = ttk.Label(key_frame, text="Ключи ещё
не сформированы.", foreground="#1f4f7a")
            self.keys_label.grid(row=1, column=0, columnspan=6,
sticky="w", pady=(8, 0))

            message_frame = ttk.LabelFrame(root, text="2. Сообщение
и данные владельца подписи", padding=10)
            message_frame.grid(row=2, column=0, sticky="ew",
pady=(0, 10))
            message_frame.columnconfigure(0, weight=1)
            ttk.Label(message_frame, text="Исходный
текст:").grid(row=0, column=0, sticky="w")
            self.message_box = tk.Text(message_frame, height=4,
wrap=tk.WORD, font=("Consolas", 10))
            self.message_box.grid(row=1, column=0, sticky="ew",
pady=(4, 8))
            self.message_box.insert("1.0", "Защита информации: RSA
подтверждает целостность сообщения.")
            fio_row = ttk.Frame(message_frame)
            fio_row.grid(row=2, column=0, sticky="ew")
            fio_row.columnconfigure(1, weight=1)
            ttk.Label(fio_row, text="ФИО для ЭЦП:").grid(row=0,
column=0, sticky="w")
            ttk.Entry(fio_row,
textvariable=self.fio_var).grid(row=0, column=1, sticky="ew",
padx=(8, 8))
            ttk.Button(fio_row, text="Зашифровать, расшифровать и
подписать", command=self.run_all).grid(row=0, column=2)

```

```

        result_frame = ttk.LabelFrame(root, text="3. Результат
выполнения", padding=10)
        result_frame.grid(row=3, column=0, sticky="nsew")
        result_frame.columnconfigure(0, weight=1)
        result_frame.rowconfigure(1, weight=1)
        ttk.Label(result_frame, textvariable=self.decrypted_var,
foreground="#176b3a").grid(row=0, column=0, sticky="w", pady=(0,
6))

        self.result_box = tk.Text(result_frame, wrap=tk.WORD,
font=("Consolas", 10), state=tk.DISABLED)
        self.result_box.grid(row=1, column=0, sticky="nsew")

        ttk.Label(
            root,
            text="Учебная модель: малые p и q выбраны для
наглядности; реальные системы используют криптографически
стойкие ключи и схемы дополнения.",
            foreground="#666666",
            wraplength=900,
        ).grid(row=4, column=0, sticky="w", pady=(10, 0))

def _read_keys(self) -> KeyPair:
    try:
        p = int(self.p_var.get().strip())
        q = int(self.q_var.get().strip())
    except ValueError as error:
        raise ValueError("p и q должны быть целыми
числами.") from error
    return build_keys(p, q)

def generate_keys(self) -> None:
    try:
        self.key_pair = self._read_keys()
    except ValueError as error:
        messagebox.showerror("Параметры RSA", str(error),
parent=self)
    return

```

```

        self.keys_label.config(
            text=(
                f"Открытый ключ: (e={self.key_pair.e},
n={self.key_pair.n}); "
                f"закрытый ключ: (d={self.key_pair.d},
n={self.key_pair.n}) "
            )
        )

def run_all(self) -> None:
    try:
        self.key_pair = self._read_keys()
        text = self.message_box.get("1.0", tk.END).strip()
        fio = self.fio_var.get().strip()
        if not fio:
            raise ValueError("Введите ФИО для формирования
ЭЦП.")

        ciphertext = encrypt_modified_rsa(text,
self.key_pair)
        decrypted = decrypt_modified_rsa(ciphertext,
self.key_pair)
        fio_digest, fio_representative, fio_signature =
sign(fio.encode("utf-8"), self.key_pair)
        cipher_payload = json.dumps(ciphertext,
separators=(",", ":")).encode("ascii")
        cipher_digest, cipher_representative,
cipher_signature = sign(cipher_payload, self.key_pair)

        self.keys_label.config(
            text=(
                f"Открытый ключ: (e={self.key_pair.e},
n={self.key_pair.n}); "
                f"закрытый ключ: (d={self.key_pair.d},
n={self.key_pair.n}) "
            )
        )

```

```

        self.decrypted_var.set(f"Расшифрованный текст:
{decrypted}")

        report = [
            "МОДИФИЦИРОВАННОЕ RSA-ШИФРОВАНИЕ",
            f"Исходный текст (UTF-8): {text}",
            f"Шифртекст ({len(ciphertext)} блоков):
{ciphertext}",
            "",
            "ЭЛЕКТРОННАЯ ПОДПИСЬ ФИО",
            f"ФИО: {fio}",
            f"SHA-256: {fio_digest}",
            f"Представитель хеша modulo n:
{fio_representative}",
            f"Подпись RSA: {fio_signature}",
            f"Проверка подписи ФИО: {'успешно' if
verify(fio.encode('utf-8'), fio_signature, self.key_pair) else
'ошибка'}",
            "",
            "ПАРА «ШИФРОВАННОЕ СООБЩЕНИЕ + ЭЦП»",
            f"SHA-256 шифртекста: {cipher_digest}",
            f"Представитель хеша modulo n:
{cipher_representative}",
            f"Подпись RSA шифртекста: {cipher_signature}",
            f"Проверка подписи шифртекста: {'успешно' if
verify(cipher_payload, cipher_signature, self.key_pair) else
'ошибка'}",
        ]

        self._set_result("\n".join(report))
    except ValueError as error:
        messagebox.showerror("Выполнение алгоритма",
str(error), parent=self)

def _set_result(self, text: str) -> None:
    self.result_box.config(state=tk.NORMAL)
    self.result_box.delete("1.0", tk.END)
    self.result_box.insert("1.0", text)
    self.result_box.config(state=tk.DISABLED)

```

```

if __name__ == "__main__":
    application = RsaLabApp()
    if "--demo" in sys.argv:
        application.after(250, application.run_all)
    application.mainloop()

```

Результат выполнения

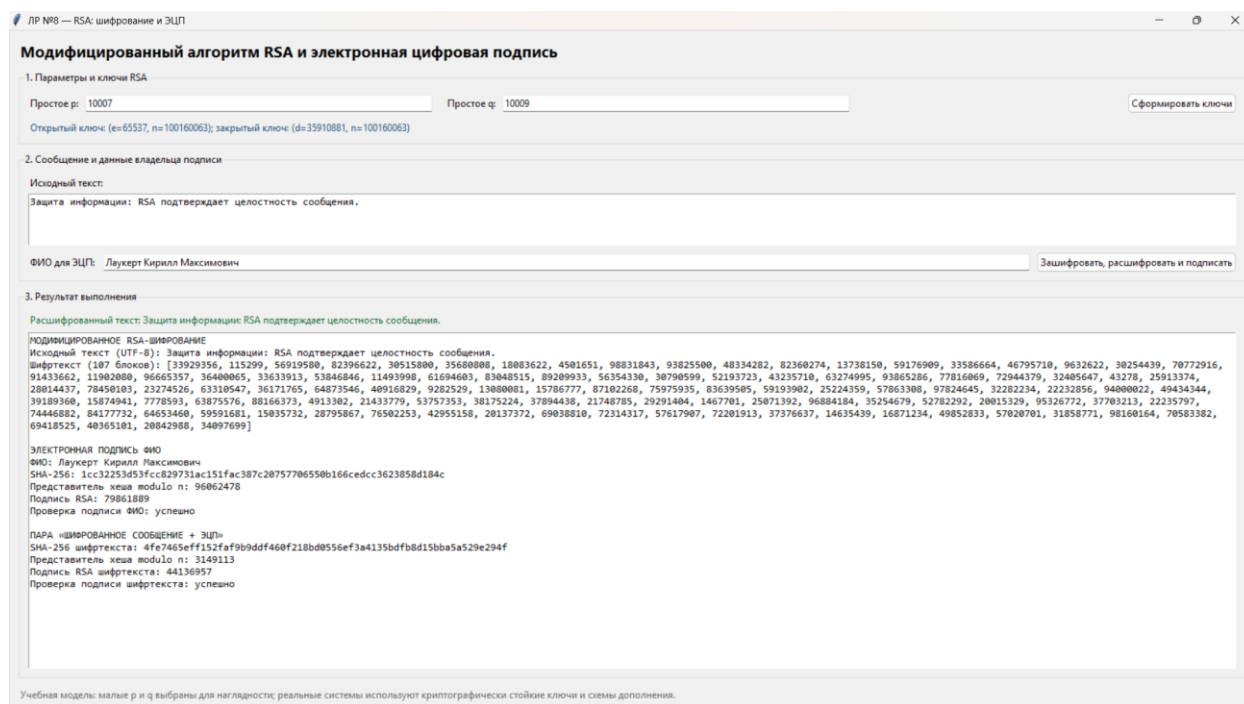


Рисунок 1 — окно приложения с результатами шифрования, расшифрования и проверки ЭЦП

Приложение успешно сформировало ключи, зашифровало исходный UTF-8 текст, восстановило его закрытым ключом, создало ЭЦП по ФИО и ЭЦП шифртекста. Обе проверки подписи завершились успешно.

Вывод

В ходе работы создано графическое приложение, реализующее модифицированный алгоритм RSA. Программа контролирует простоту p и q , выводит пару ключей, использует случайный начальный элемент цепочки при шифровании и корректно восстанавливает исходный текст. Реализованы цифровые подписи для ФИО и шифрованного сообщения с проверкой их целостности и подлинности.

Список использованных источников

1. Панасенко С. Алгоритмы шифрования. СПб.: БХВ-Петербург, 2009.

2. Рябко Б. Я., Фионов А. Н. Основы современной криптографии для специалистов в информационных технологиях. М.: Научный мир, 2004.
3. Документация Python: hashlib — Secure hashes and message digests; tkinter — Python interface to Tcl/Tk.